

Discrete Structure

Automated Logical Reasoning System (ALRS)

1. Project Overview:

The Automated Logical Reasoning System (ALRS) is a pure procedural C++ program designed to parse, evaluate, and validate complex logical expressions. The system was built without any Object-Oriented Programming (OOP) features or the Standard Template Library (STL), relying solely on arrays, loops, and custom logic implementation.

2. System Architecture & Module Descriptions:

The project is structured into modular functions, each handling a specific stage of the logic processing pipeline.

Core Logic Modules

1. Tokenizer (Function: tokenization)

Purpose: Preprocesses the user's raw input string.

Logic: It iterates through the input character array, removing whitespace and validating that each character is a legal symbol (variable A-Z, operator, or parenthesis).

Output: A clean, compact character array ready for parsing.

2. Parser (Function: infixToPostfix)

Purpose: Converts human-readable Infix expressions (e.g., "P & Q") into machine-readable Postfix notation (e.g., "PQ&").

Algorithm: The Shunting-yard algorithm.

Implementation details:

- ❑ Variables (P, Q) are sent directly to the output string.
- ❑ Operators (!, &, |, >, =) are pushed onto a custom char stack (stack[100]).
- ❑ Operator precedence is strictly enforced using a helper function Precedence(). A lowerprecedence operator cannot be placed on top of a higher-precedence one; the stack is popped first.
- ❑ Parentheses () are handled by pushing (to the stack and popping everything until) is found when closing a group.

3. Evaluator (Function: Evaluator)

Purpose: Calculates the Truth Value (1 or 0) of a postfix expression.

Logic:

- ❑ It uses a custom integer stack (ValueStack[100]).
- ❑ It scans the postfix string left-to-right.

- If a Variable is found, it looks up its truth value (0/1) from the current truth assignment array and pushes it.
- If an Operator is found, it pops the required operands (1 for NOT, 2 for others), performs the logical operation (AND, OR, IMPLIES, BICONDITIONAL), and pushes the result back.

Critical Logic: The stack pop order was carefully managed (Right Operand first, Left Operand second) to ensure correct implication logic ($A > B$ vs $B > A$).

Feature Modules (Menu Options)

4. Truth Table Generator (Choice 1):

Functionality: Generates a full truth table for any expression.

Logic:

- It detects unique variables (P, Q, R) using a frequency array (seen[256]).
- It calculates the total rows required as 2^N (using pow).
- It generates truth values using a binary counting loop (Odometer method). For row i and variable j , the truth value is calculated as $(i / 2^{(N-1-j)}) \% 2$.
- The Evaluator is called for every single row to determine the final Result column.

5. Argument Validator (Choice 2):

Functionality: Validates if a set of Premises logically implies a Conclusion.

Logic:

- It accepts multiple premises and a conclusion from the user.
- It iterates through all 2^N truth combinations.
- For each row, it checks: "Are ALL premises True?"

Logic Rule: If (All Premises == True) AND (Conclusion == False), the argument is INVALID.

If such a row exists, it is reported as a counterexample. Otherwise, the argument is VALID.

6. Equivalence Checker (Choice 3)

Functionality: Checks if two different logical expressions are mathematically identical.

Logic:

- It generates truth tables for both Expression A and Expression B simultaneously.
- It compares the Result of Evaluator(A) vs Evaluator(B) for every single row.
- If a mismatch is found, it immediately reports "Not Equivalent." If the loop finishes with no mismatches, they are "Equivalent."

7. Reasoning Chain (Choice 4)

Functionality: Simulates a reasoning chain (e.g., Modus Ponens).

Logic:

- It accepts user inputs for variables (e.g., P and Q).
- It automatically constructs a logical implication chain string, such as $((P \rightarrow Q) \wedge P) \rightarrow Q$ using `streat`.
- It runs this constructed chain through the Validator engine. If the chain evaluates to True for all possible assignments (a Tautology), the reasoning is confirmed as Valid.

8. File Persistence & Viewer (Choice 5)

Functionality: Allows the user to view previously saved results and persists data between operations.

Saving Logic: A helper function `saveResult` is called only after the system prompts the user ("Do you want to save results? Y/N") and receives confirmation, ensuring user consent for data persistence.

Viewing Logic: Uses `ifstream` to open `results.txt` and reads the file line-by-line using a while loop, printing the history to the console.

Challenges & SOLUTIONS**9. Manual Memory Management:**

Challenge: C++ strings (`std::string`) were forbidden.

Solution: Used fixed-size char arrays (e.g., `char exp[100]`) and manual null-termination (`\0`) to handle text safely.

10. The "Shunting-Yard" Implementation:

Challenge: Handling operator precedence manually without a map or built-in stack.

Solution: Created a custom `Precedence()` helper function that assigns integer weights (e.g., NOT=4, AND=3, OR=2) to guide the stack operations.

11. Logic Validation Loops:

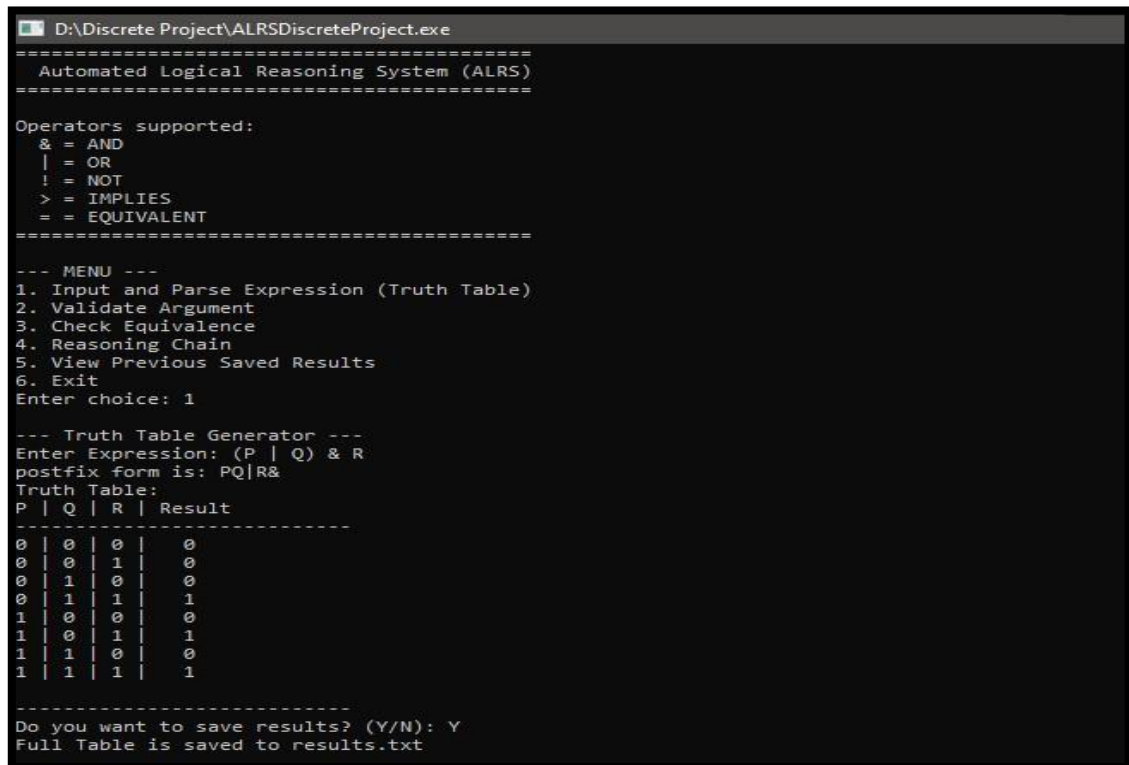
Challenge: Ensuring variables were consistently mapped across multiple premises.

Solution: Implemented a robust variable detection loop that scans ALL inputs first to build a single master list of variables (P, Q, R) before generating the truth table.

Sample Execution Screenshots

1: Truth Table Output:

- Shows the system generating a full table for $(P|Q) \& R$.



```
D:\Discrete Project\ALRSDiscreteProject.exe
=====
Automated Logical Reasoning System (ALRS)
=====

Operators supported:
& = AND
| = OR
! = NOT
> = IMPLIES
= = EQUIVALENT
=====

--- MENU ---
1. Input and Parse Expression (Truth Table)
2. Validate Argument
3. Check Equivalence
4. Reasoning Chain
5. View Previous Saved Results
6. Exit
Enter choice: 1

--- Truth Table Generator ---
Enter Expression: (P | Q) & R
postfix form is: PQ|R&
Truth Table:
P | Q | R | Result
-----
0 | 0 | 0 | 0
0 | 0 | 1 | 0
0 | 1 | 0 | 0
0 | 1 | 1 | 1
1 | 0 | 0 | 0
1 | 0 | 1 | 1
1 | 1 | 0 | 0
1 | 1 | 1 | 1

-----
Do you want to save results? (Y/N): Y
Full Table is saved to results.txt
```

2: Argument Validator Output

- Shows the system validating a valid argument (Modus Ponens).

```

--- MENU ---
1. Input and Parse Expression (Truth Table)
2. Validate Argument
3. Check Equivalence
4. Reasoning Chain
5. View Previous Saved Results
6. Exit
Enter choice: 2

--- Argument Validator ---
Number of Premises: 2
premise:1
inputP>Q
premise:2
inputP
Conclusion:
inputQ

Checking...
Result:Valid Argument
Do you want to save results? (Y/N): Y
[Saved to results.txt]

```

3: Equivalence Checker Output

- Shows the system proving De Morgan's Law.

```

--- MENU ---
1. Input and Parse Expression (Truth Table)
2. Validate Argument
3. Check Equivalence
4. Reasoning Chain
5. View Previous Saved Results
6. Exit
Enter choice: 3

--- Equivalence Checker ---
Enter Expression 1: !(P & Q)
Enter Expression 2: !P | !Q
Result:Equivalent!Identical logic.
Do you want to save results? (Y/N): N

```

4: Reasoning Chain Output

- Shows the system validating a reasoning chain for P and Q.

```

--- MENU ---
1. Input and Parse Expression (Truth Table)
2. Validate Argument
3. Check Equivalence
4. Reasoning Chain
5. View Previous Saved Results
6. Exit
Enter choice: 4

--- Reasoning Chain ---
Checks if premise to Result is valid logic. Enter Premise Variable: P
Enter Result Variable: Q
Validating Chain: ((P>Q)&P)>Q
Result: Valid Reasoning Chain!
Do you want to save results? (Y/N): Y
[Saved to results.txt]

```

5: File Viewer Output:

- Shows the system reading back the saved history from results.txt.

```

--- MENU ---
1. Input and Parse Expression (Truth Table)
2. Validate Argument
3. Check Equivalence
4. Reasoning Chain
5. View Previous Saved Results
6. Exit
Enter choice: 5

--- Saved Results ---

Truth Table:
P | Q | R | Result
-----
0 | 0 | 0 | 0
0 | 0 | 1 | 0
0 | 1 | 0 | 0
0 | 1 | 1 | 1
1 | 0 | 0 | 0
1 | 0 | 1 | 1
1 | 1 | 0 | 0
1 | 1 | 1 | 1
-----

Argument Validator: Valid
Reasoning Chain: Valid

```